

## Лабораторная работа № 1

### Предобработка данных: устранение пропусков, кодирование категориальных признаков и нормализация числовых признаков

Датасет: **Heart Disease UCI** (Данные о сердечно-сосудистых заболеваниях)

### Описание набора данных

Для выполнения лабораторной работы был выбран датасет Heart Disease UCI, доступный на платформе Kaggle (<https://www.kaggle.com/datasets/ronitf/heart-disease-uci>). Набор данных содержит медицинские показатели пациентов и используется для задач бинарной классификации наличия сердечно-сосудистых заболеваний.

Датасет содержит 920 записи и 16 признаков, среди которых присутствуют как числовые, так и категориальные признаки, а также пропуски в данных. Удалим 2 признака – id и dataset, так как они не несут никакой полезной информации, оставим 14

### Характеристика признаков

| Признак  | Тип              | Описание                                                        |
|----------|------------------|-----------------------------------------------------------------|
| age      | Числовой (int)   | Возраст пациента                                                |
| sex      | Категориальный   | Пол (0 — женщина, 1 — мужчина)                                  |
| cp       | Категориальный   | Тип боли в груди (0–3)                                          |
| trestbps | Числовой (int)   | Артериальное давление в покое (мм рт. ст.)                      |
| chol     | Числовой (int)   | Уровень холестерина (мг/дл)                                     |
| fbs      | Категориальный   | Сахар крови натощак > 120 мг/дл (1 = да, 0 = нет)               |
| restecg  | Категориальный   | Результаты ЭКГ покоя (0–2)                                      |
| thalch   | Числовой (int)   | Максимальная частота сердечных сокращений                       |
| exang    | Категориальный   | Стенокардия при нагрузке (1 = да, 0 = нет)                      |
| oldpeak  | Числовой (float) | Депрессия ST при нагрузке относительно покоя                    |
| slope    | Категориальный   | Наклон сегмента ST при нагрузке (0–2)                           |
| ca       | Числовой (int)   | Количество крупных сосудов (0–3), пропуски                      |
| thal     | Категориальный   | Тип дефекта (normal, fixed defect, reversible defect), пропуски |

|     |         |                          |
|-----|---------|--------------------------|
| num | Целевой | Предсказываемое значение |
|-----|---------|--------------------------|

## Импорт библиотек и загрузка данных

В начале работы выполняется импорт необходимых библиотек и загрузка набора данных:

Пример первых строк датасета (до предобработки):

|   | age | sex    | cp              | trestbps | chol  | fbs   | restecg        | thalch | exang | oldpeak | slope       | ca  | thal              | num |
|---|-----|--------|-----------------|----------|-------|-------|----------------|--------|-------|---------|-------------|-----|-------------------|-----|
| 0 | 63  | Male   | typical angina  | 145.0    | 233.0 | True  | lv hypertrophy | 150.0  | False | 2.3     | downsloping | 0.0 | fixed defect      | 0   |
| 1 | 67  | Male   | asymptomatic    | 160.0    | 286.0 | False | lv hypertrophy | 108.0  | True  | 1.5     | flat        | 3.0 | normal            | 2   |
| 2 | 67  | Male   | asymptomatic    | 120.0    | 229.0 | False | lv hypertrophy | 129.0  | True  | 2.6     | flat        | 2.0 | reversable defect | 1   |
| 3 | 37  | Male   | non-anginal     | 130.0    | 250.0 | False | normal         | 187.0  | False | 3.5     | downsloping | 0.0 | normal            | 0   |
| 4 | 41  | Female | atypical angina | 130.0    | 204.0 | False | lv hypertrophy | 172.0  | False | 1.4     | upsloping   | 0.0 | normal            | 0   |

## Типы данных и пропуски

Перед предобработкой необходимо изучить типы признаков и наличие пропусков:

```
data.dtypes
```

Вывод команды data.dtypes:

| Признак  | Тип данных |
|----------|------------|
| age      | int64      |
| sex      | object     |
| cp       | object     |
| trestbps | float64    |
| chol     | float64    |
| fbs      | object     |
| restecg  | object     |
| thalch   | float64    |
| exang    | object     |
| oldpeak  | float64    |
| slope    | object     |
| ca       | float64    |

|             |        |
|-------------|--------|
| <b>thal</b> | object |
| <b>num</b>  | int64  |

Подсчёт пропусков:

```
data.isnull().sum()
```

| Признак         | Количество пропусков |
|-----------------|----------------------|
| <b>age</b>      | 0                    |
| <b>sex</b>      | 0                    |
| <b>cp</b>       | 0                    |
| <b>trestbps</b> | 59                   |
| <b>chol</b>     | 30                   |
| <b>fbs</b>      | 90                   |
| <b>restecg</b>  | 2                    |
| <b>thalch</b>   | 55                   |
| <b>exang</b>    | 55                   |
| <b>oldpeak</b>  | 62                   |
| <b>slope</b>    | 309                  |
| <b>ca</b>       | 611                  |
| <b>thal</b>     | 486                  |
| <b>num</b>      | 0                    |

## Устранение пропусков в данных

Пропуски в датасете присутствуют во многих столбцах, основные:

|              |     |
|--------------|-----|
| <b>slope</b> | 309 |
| <b>ca</b>    | 611 |
| <b>thal</b>  | 486 |

## Удаление столбцов с пропусками

Самый простой способ — удалить все столбцы, содержащие хотя бы один пропуск. Применяется метод `dropna` с параметром `axis=1`:

## 1) Удаление колонок, содержащие пустые значения

```
data_new_1 = data.dropna(axis=1, how='any')  
(data.shape, data_new_1.shape)
```

```
((920, 14), (920, 4))
```

```
data_new_1.columns
```

```
Index(['age', 'sex', 'cp', 'num'], dtype='object')
```

```
data_new_1.head()
```

|   | age | sex    | cp              | num |
|---|-----|--------|-----------------|-----|
| 0 | 63  | Male   | typical angina  | 0   |
| 1 | 67  | Male   | asymptomatic    | 2   |
| 2 | 67  | Male   | asymptomatic    | 1   |
| 3 | 37  | Male   | non-anginal     | 0   |
| 4 | 41  | Female | atypical angina | 0   |



Результат сократился с 14 до 4 признаков. Этот метод прост, но приводит к большой потере потенциально полезных признаков.

## Удаление строк с пропусками

Альтернативный подход — удаление строк, содержащих пропуски:

```
▶ data_new_2 = data.dropna(axis=0, how='any')  
(data.shape, data_new_2.shape)
```

```
... ((920, 14), (299, 14))
```

Результат: удалено тоже очень много строк ( больше половины)

Найдем пропуски в процентном соотношении

age - 0%

sex - 0%

cp - 0%

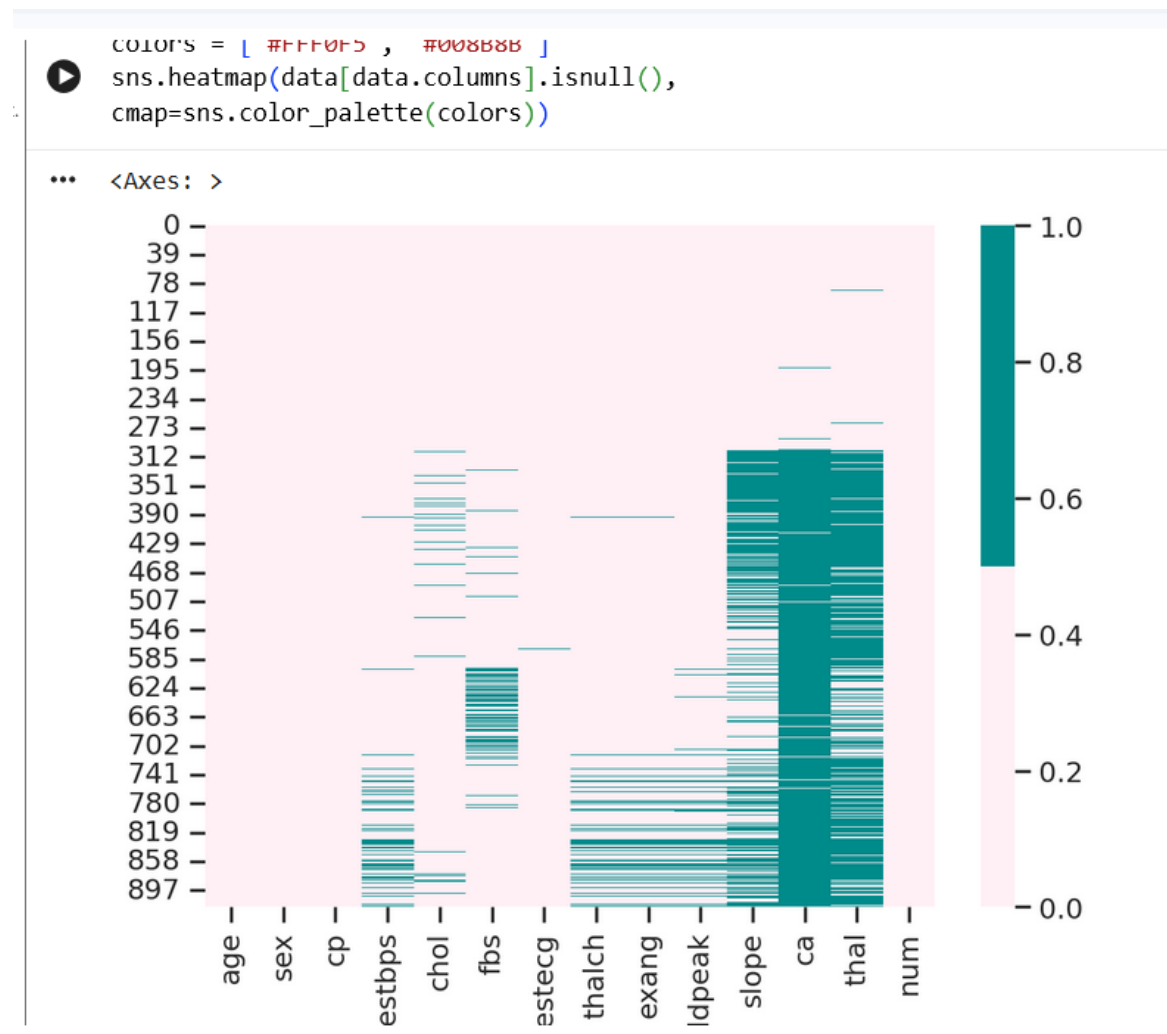
trestbps - 6%

chol - 3%

fbs - 10%

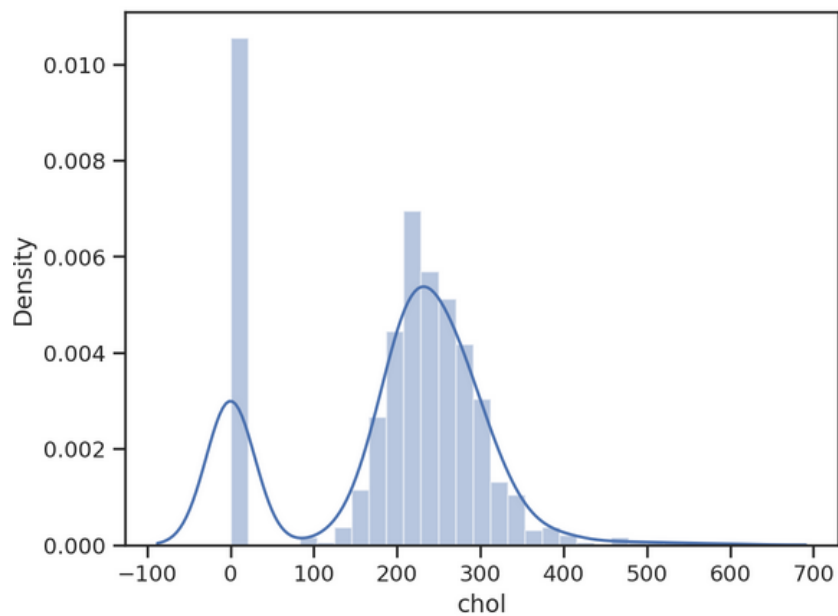
restecg - 0%

thalch - 6%  
exang - 6%  
oldpeak - 7%  
slope - 34%  
ca - 66%  
thal - 53%  
num - 0%



### Заполнение пропусков нулями

```
sns.distplot(data['chol'].fillna(0))
```

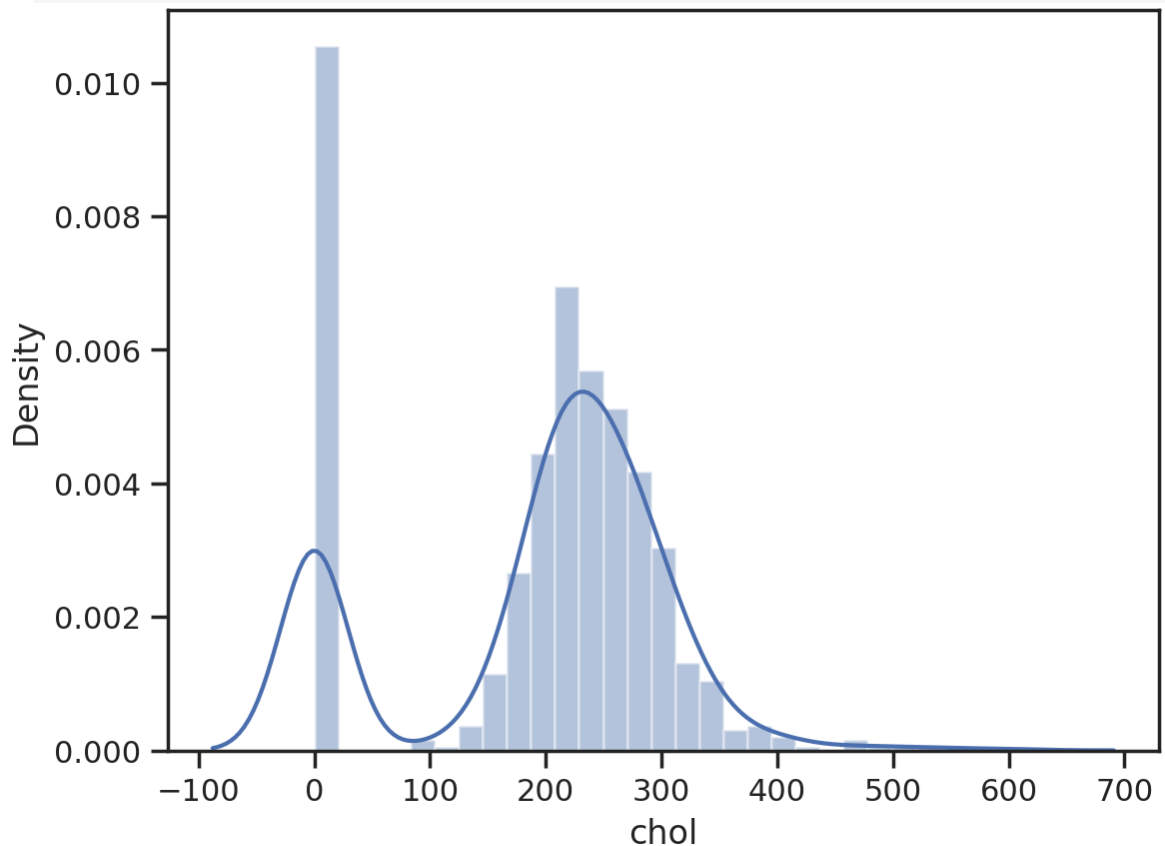


Получаем не совсем то, что хотелось

**Для удобства остальное будем выполнять в Google Collab, экспортируя результаты в пдф для отчета**

## 1) Заполнение пропусков нулями

```
In [64]: sns.distplot(data['chol'].fillna(0))
```



Получаем совершенно не то, что нам нужно С помощью класса SimpleImputer можно проводить импьютацию различными показателями центра распределения

```
In [70]:
```

Во избежание ошибки из-за булевых значений, изменим их на int

```
In [75]: data['fbs'] = data['fbs'].fillna(False).astype(int)
data['fbs'] = data['fbs'].fillna(True).astype(int)
data_new_2['fbs'] = data_new_2['fbs'].fillna(True).astype(int)
data_new_2['fbs'] = data_new_2['fbs'].fillna(False).astype(int)
data['exang'] = data['exang'].fillna(False).astype(int)
data['exang'] = data['exang'].fillna(True).astype(int)
data_new_2['exang'] = data_new_2['exang'].fillna(True).astype(int)
data_new_2['exang'] = data_new_2['exang'].fillna(False).astype(int)
```

```

/tmp/ipykernel_1352/13880586.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    data_new_2['fbs'] = data_new_2['fbs'].fillna(True).astype(int)
/tmp/ipykernel_1352/13880586.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    data_new_2['fbs'] = data_new_2['fbs'].fillna(False).astype(int)
/tmp/ipykernel_1352/13880586.py:5: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
    data['exang'] = data['exang'].fillna(False).astype(int)
/tmp/ipykernel_1352/13880586.py:7: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
    data_new_2['exang'] = data_new_2['exang'].fillna(True).astype(int)
/tmp/ipykernel_1352/13880586.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    data_new_2['exang'] = data_new_2['exang'].fillna(True).astype(int)
/tmp/ipykernel_1352/13880586.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    data_new_2['exang'] = data_new_2['exang'].fillna(False).astype(int)

```

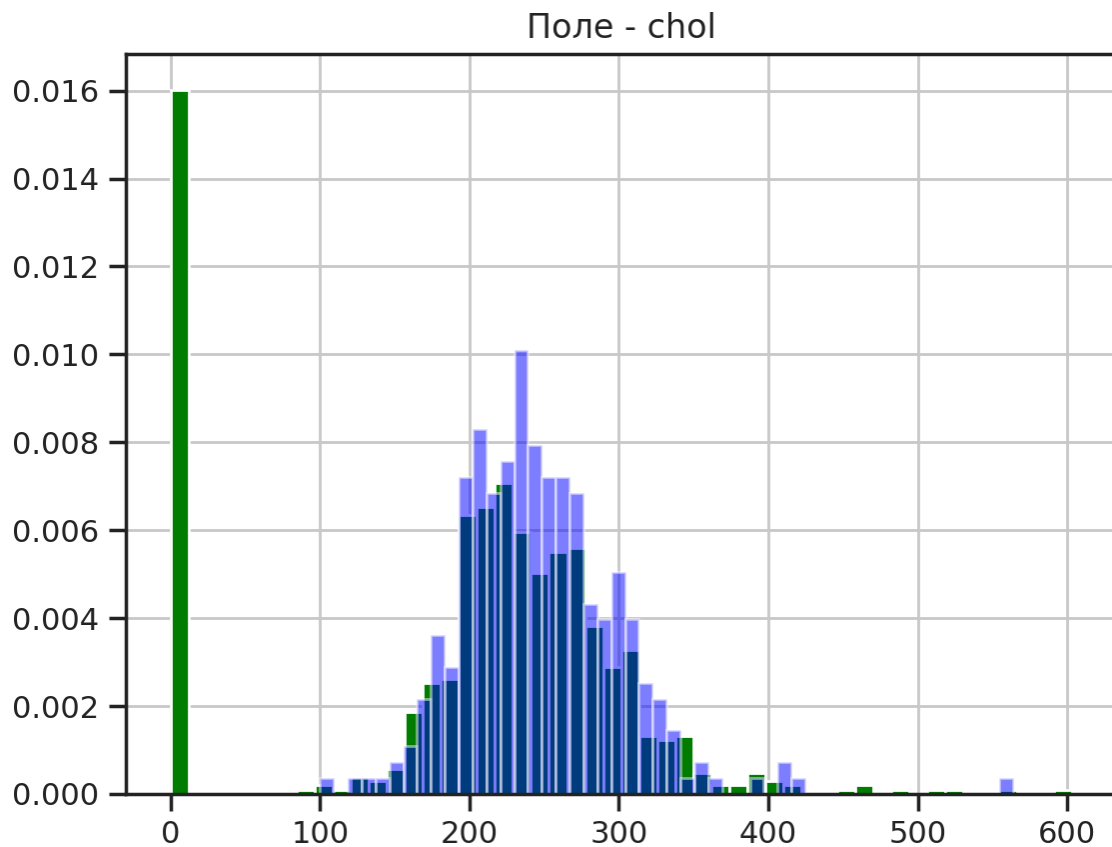
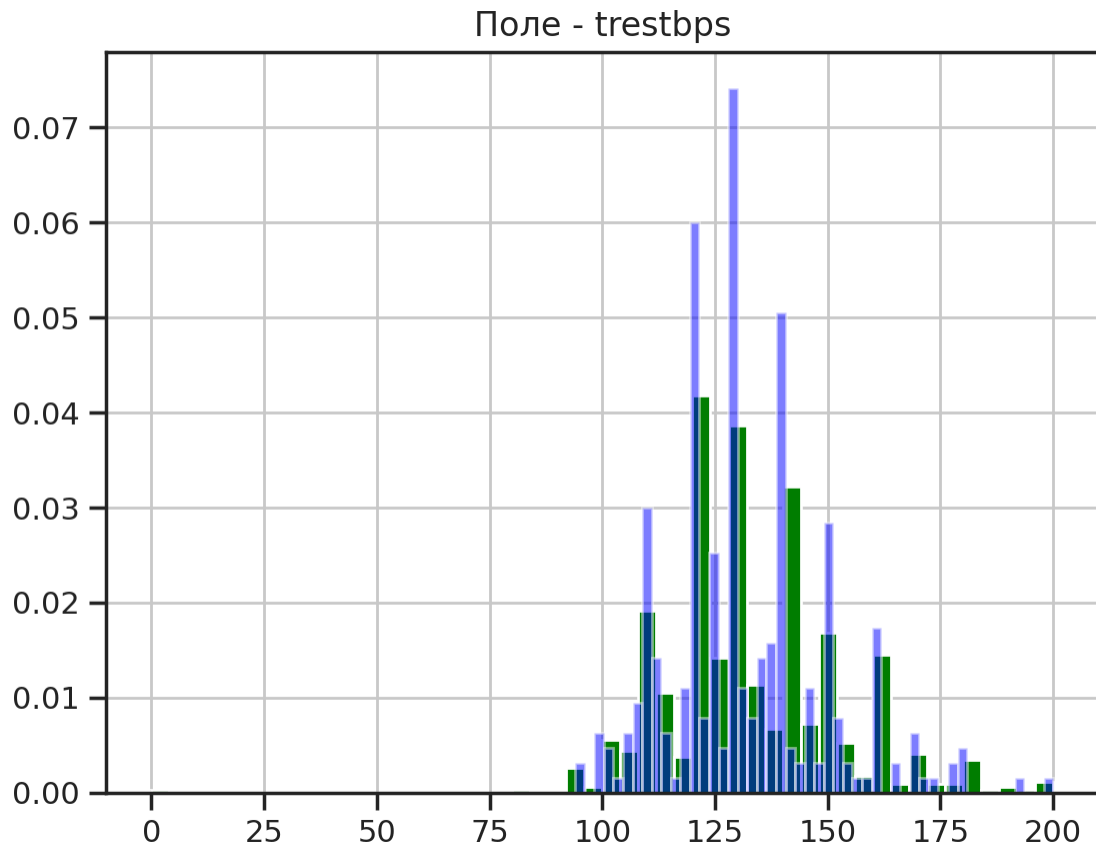
```

In [77]: def plot_hist_diff(old_ds, new_ds, cols):
        """
        Разница между распределениями до и после устранения пропусков
        """
        for c in cols:
            fig = plt.figure()
            ax = fig.add_subplot(111)
            ax.title.set_text('Поле - ' + str(c))
            old_ds[c].hist(bins=50, ax=ax, density=True, color='green')
            new_ds[c].hist(bins=50, ax=ax, color='blue', density=True, alpha=0.5)
            plt.show()

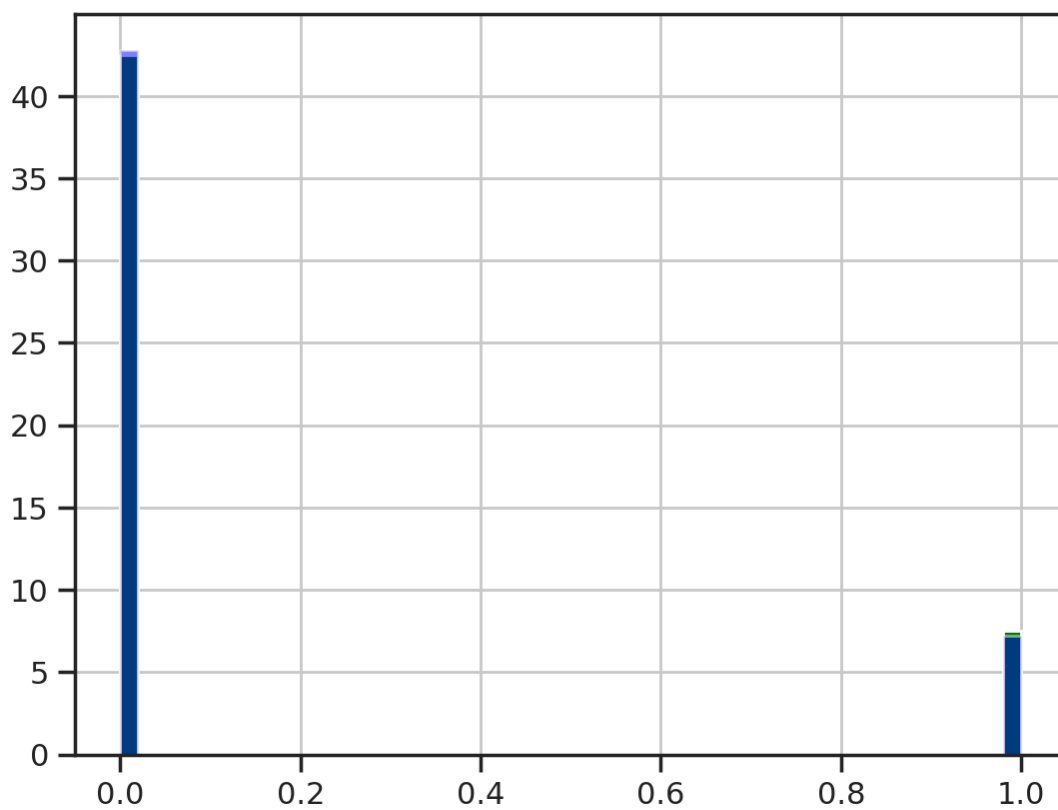
In [79]: # Колонки для которых удаляются пропуски
        hcols_with_na_temp = ['trestbps', 'chol', 'fbs', 'thalch', 'exang', 'oldpeak', 's

```

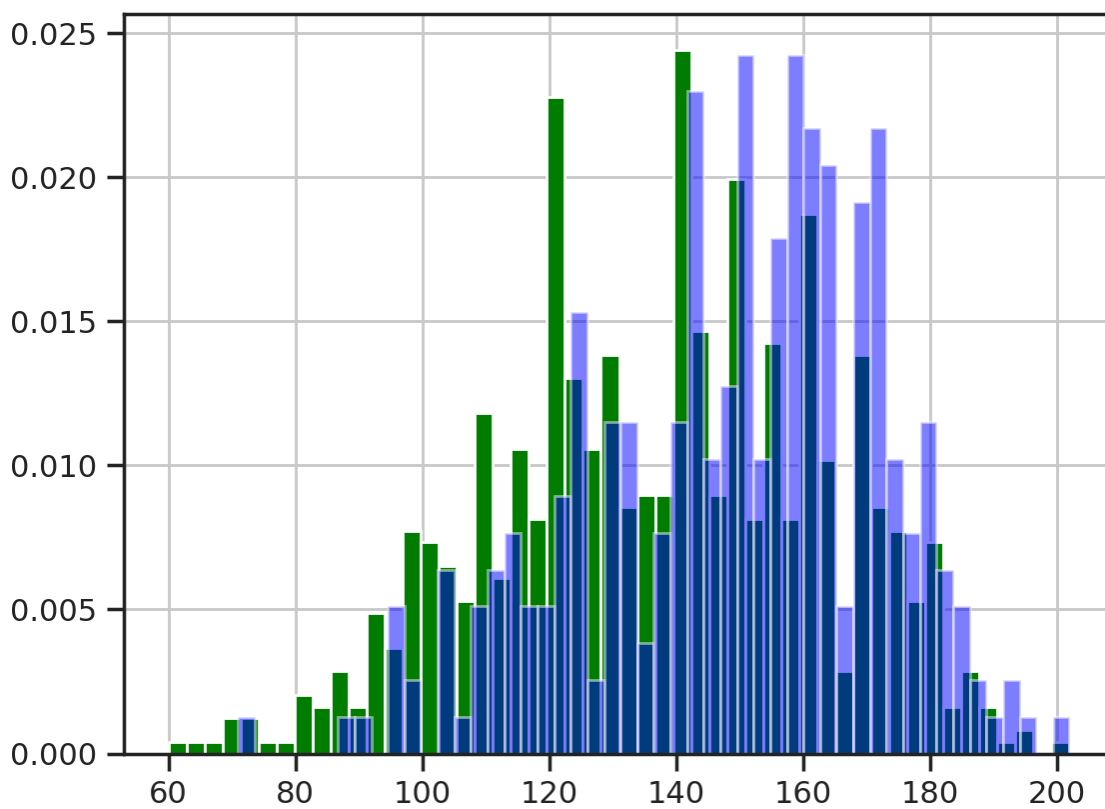
```
plot_hist_diff(data, data_new_2, hcols_with_na_temp)
```

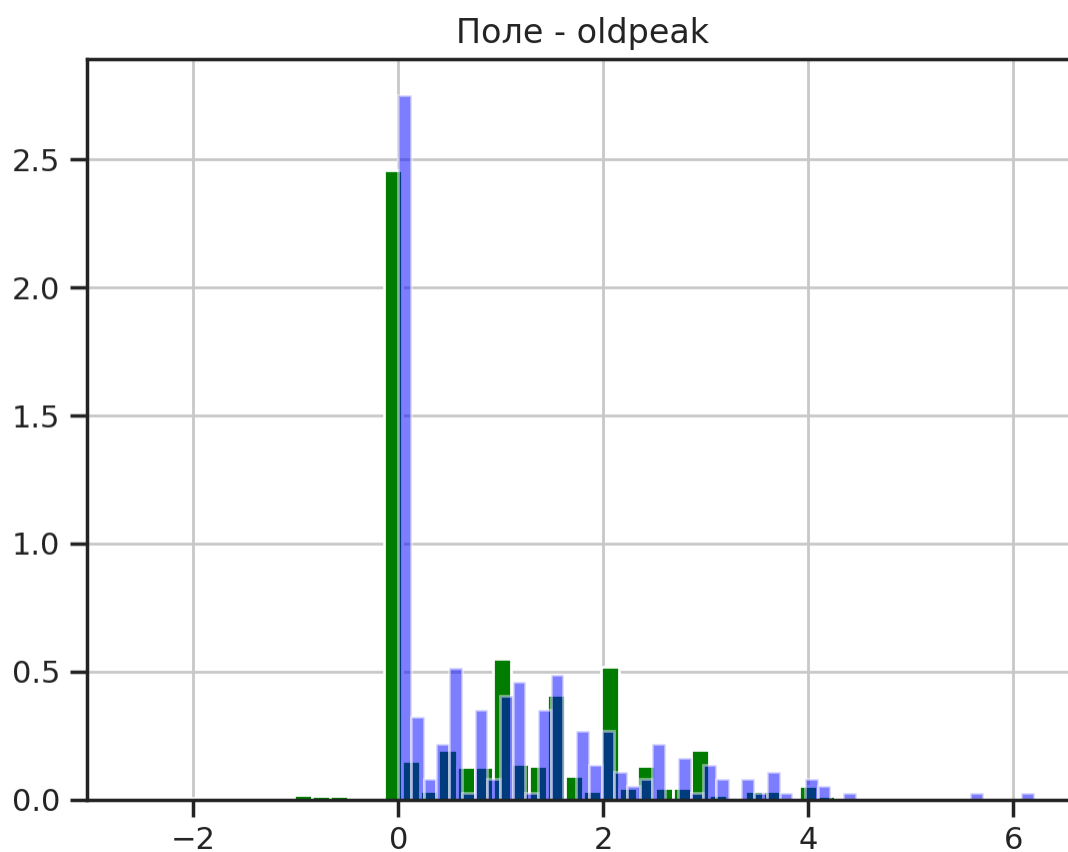
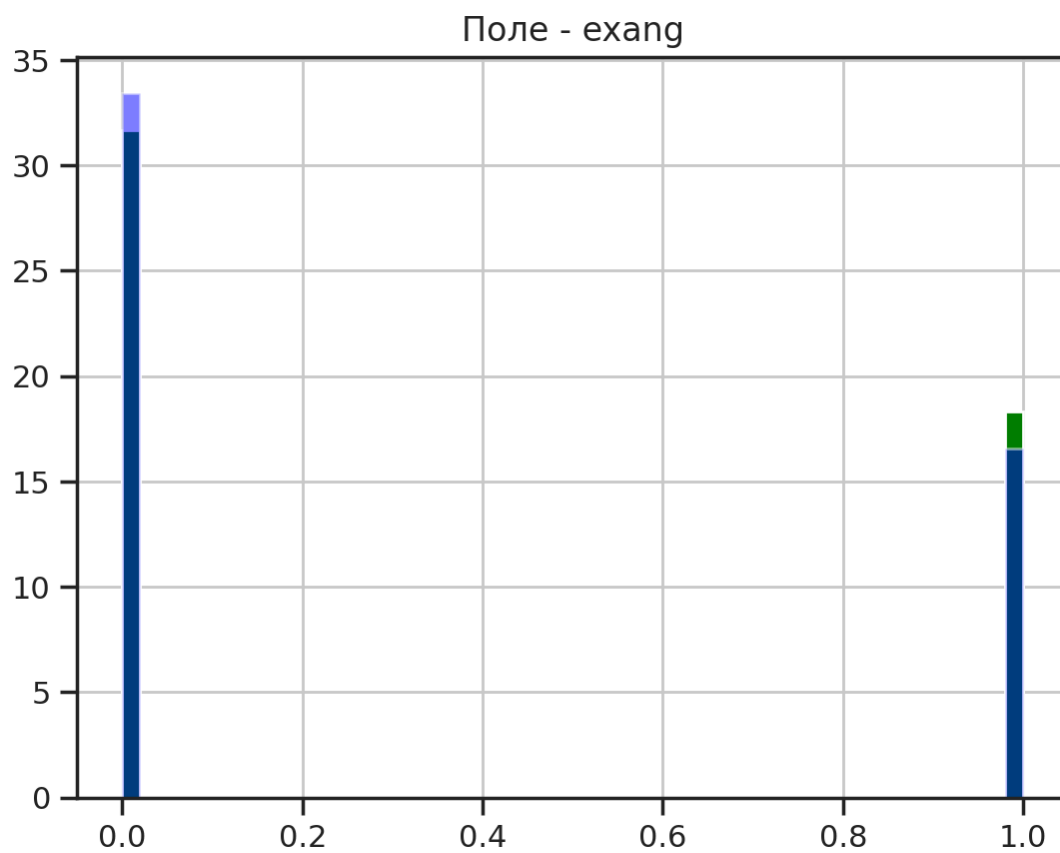


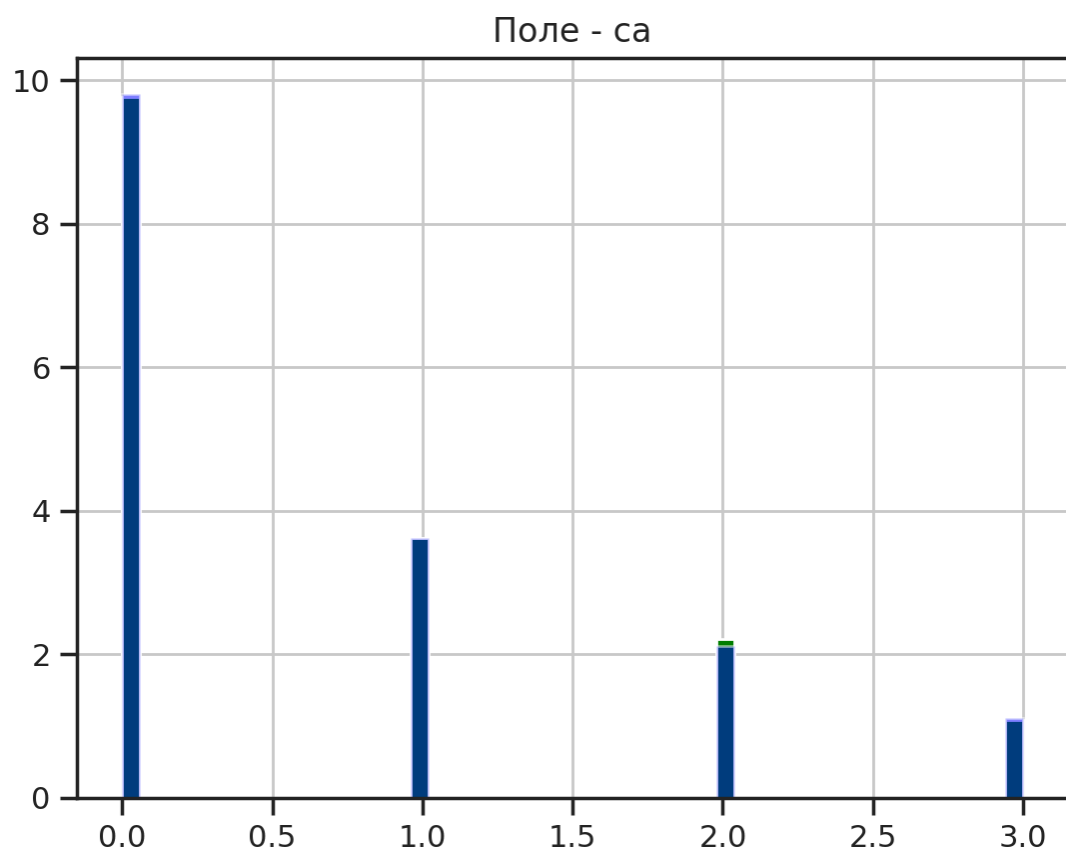
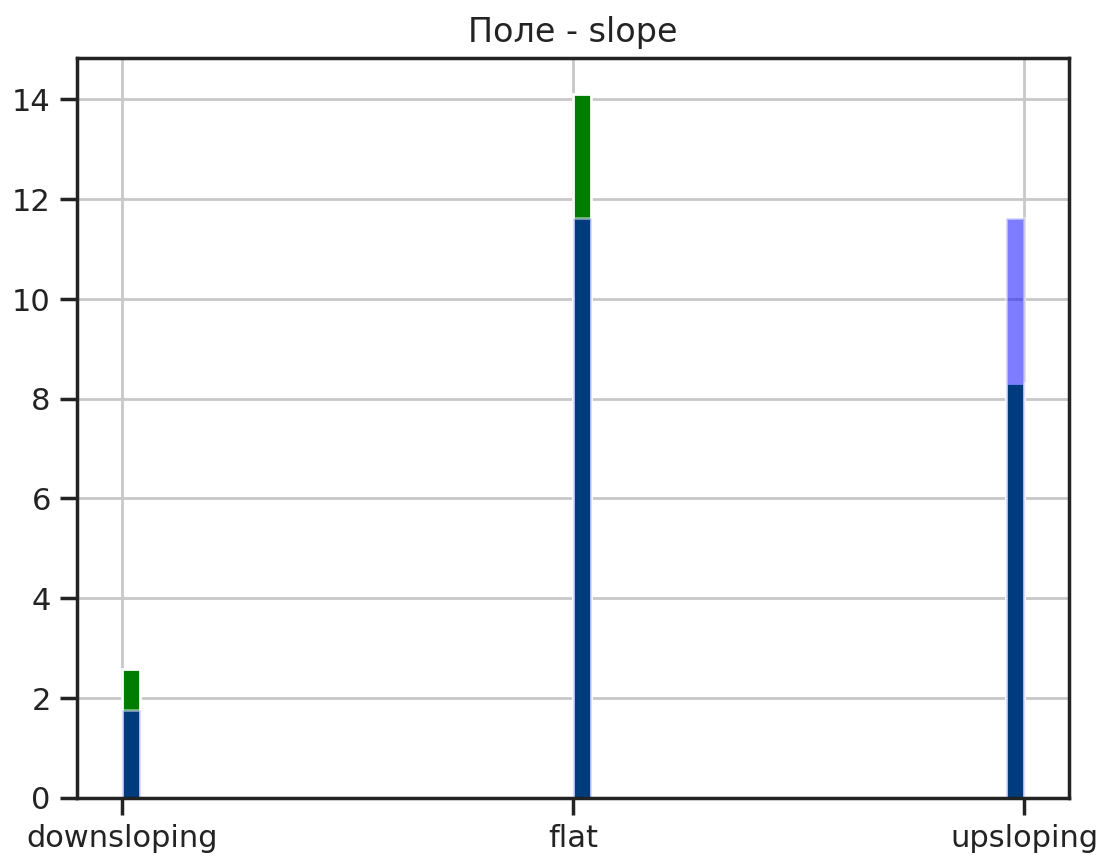
Поле - fbs

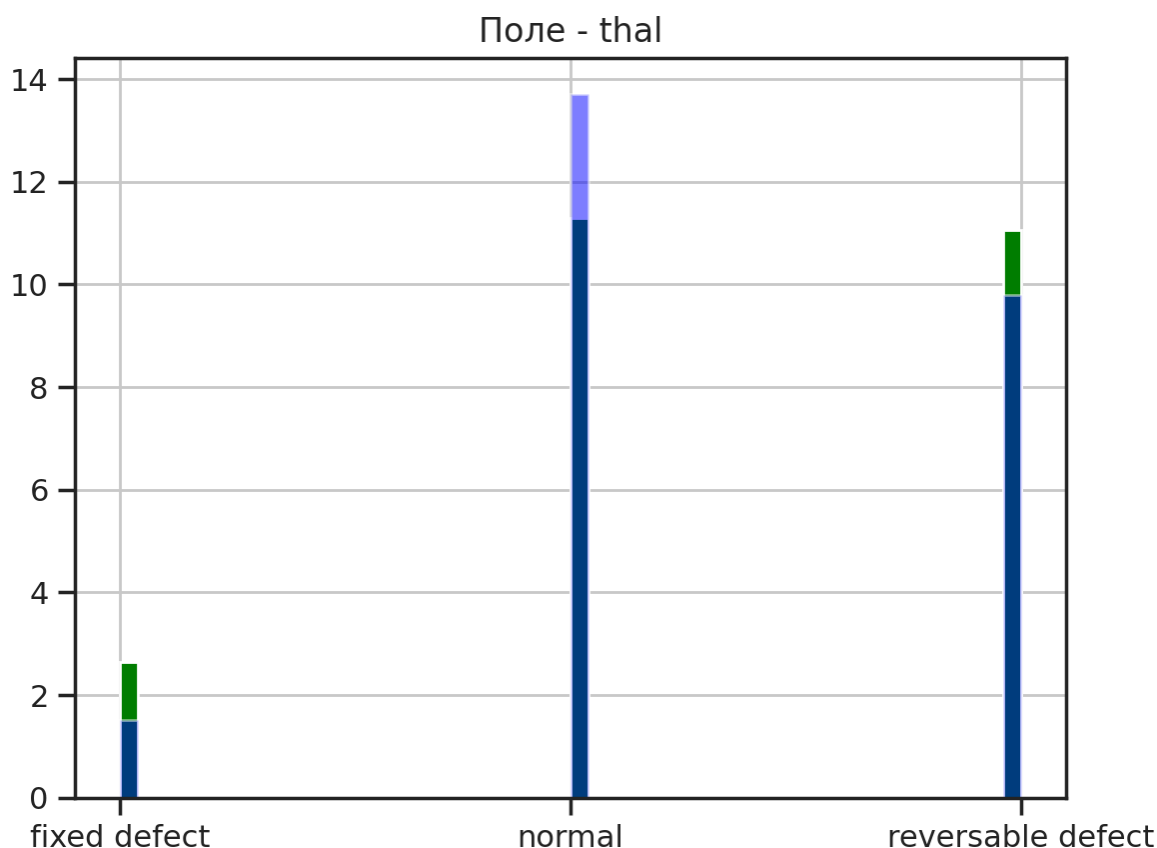


Поле - thalch









## 2) Применим заполнение средними значениями

Получается хороший результат

```
In [80]: mean=sklearn.impute.SimpleImputer(strategy="mean") # Среднее значение
mean_rate=mean.fit_transform(data[['thalch']])
sns.distplot(mean_rate)
```

/tmp/ipykernel\_1352/1255938053.py:3: UserWarning:

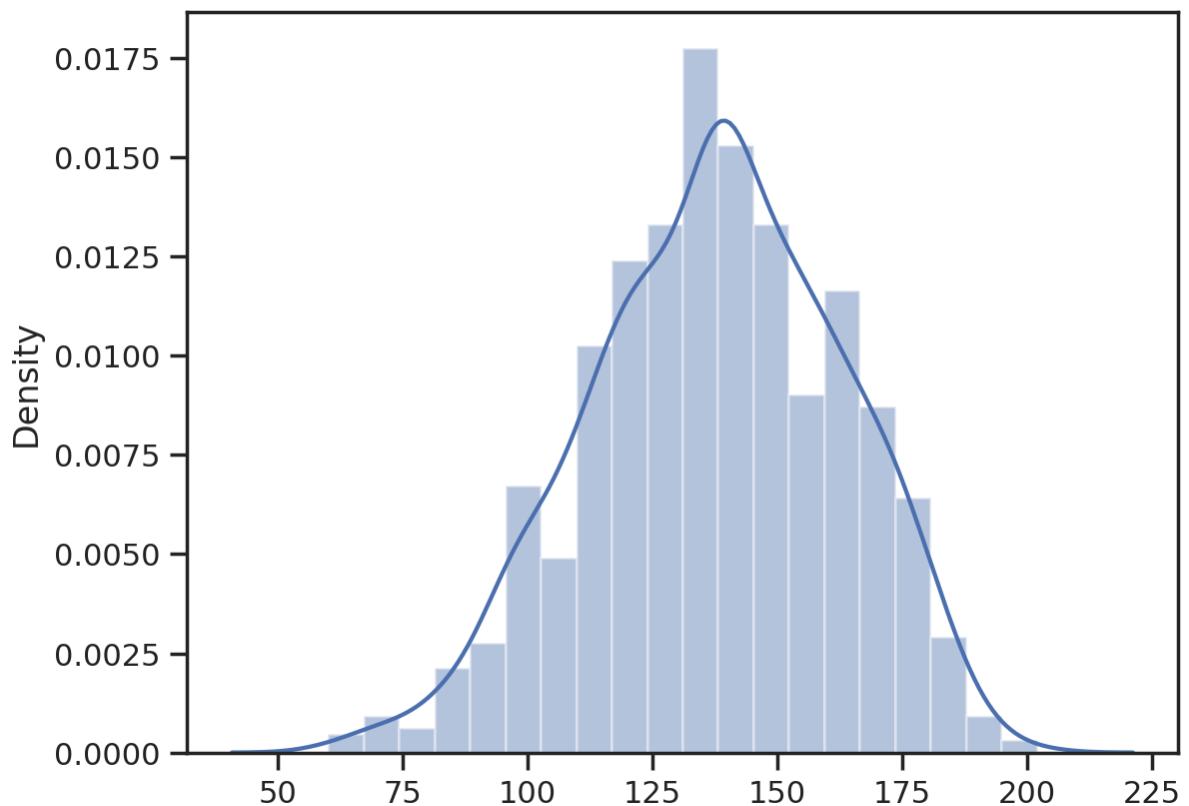
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(mean_rate)
```

Out[80]: <Axes: ylabel='Density'>



### 3) Применение медианного значения

```
In [81]: med=sklearn.impute.SimpleImputer(strategy="median") # Медиана
med_rate=med.fit_transform(data[['thalch']])
sns.distplot(med_rate)
```

/tmp/ipykernel\_1352/1137626058.py:3: UserWarning:

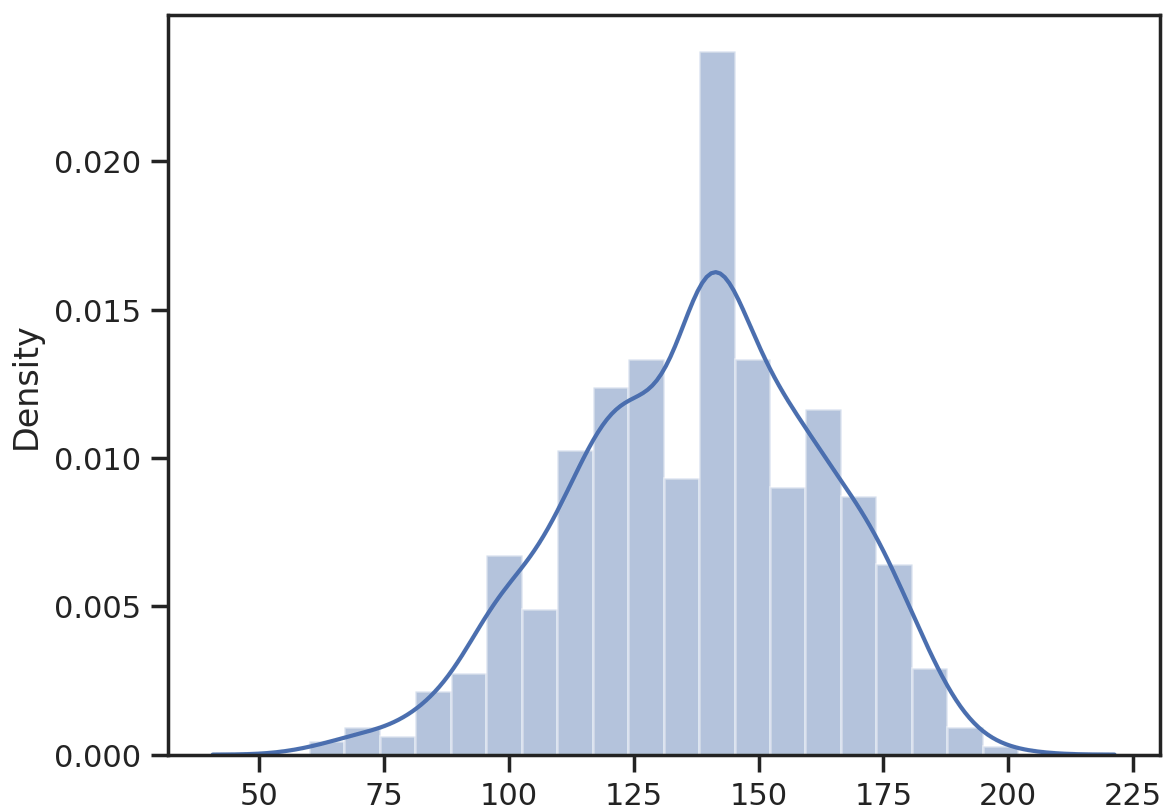
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(med_rate)
```

Out[81]: <Axes: ylabel='Density'>



#### 4) Сравнение сразу нескольких стратегии заполнения данных

```
In [82]: from sklearn.impute import MissingIndicator
from sklearn.impute import SimpleImputer

def impute_column(dataset, column, strategy_param, fill_value_param=None):
    """
    Заполнение пропусков в одном признаке
    """
    temp_data = dataset[[column]].values
    size = temp_data.shape[0]

    indicator = MissingIndicator()
    mask_missing_values_only = indicator.fit_transform(temp_data)

    imputer = SimpleImputer(strategy=strategy_param, fill_value=fill_value_param)
    all_data = imputer.fit_transform(temp_data)

    missed_data = temp_data[mask_missing_values_only]
    filled_data = all_data[mask_missing_values_only]

    return all_data.reshape((size,)), filled_data, missed_data
```

```
In [38]: def research_impute_numeric_column(dataset, num_column, const_value=None):
```

```

strategy_params = ['mean', 'median', 'most_frequent', 'constant']
strategy_params_names = ['Среднее', 'Медиана', 'Мода']
strategy_params_names.append('Константа = ' + str(const_value))

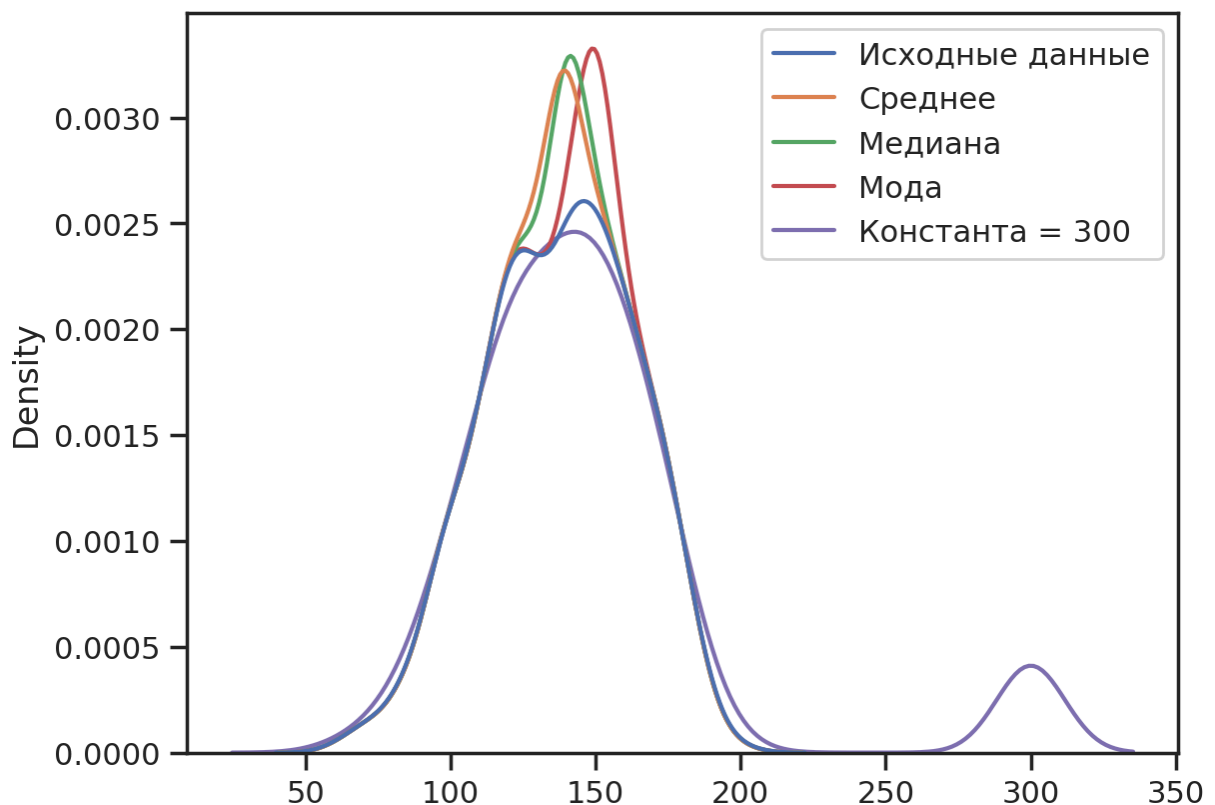
original_temp_data = dataset[[num_column]].values
size = original_temp_data.shape[0]
original_data = original_temp_data.reshape((size,))

new_df = pd.DataFrame({'Исходные данные':original_data})

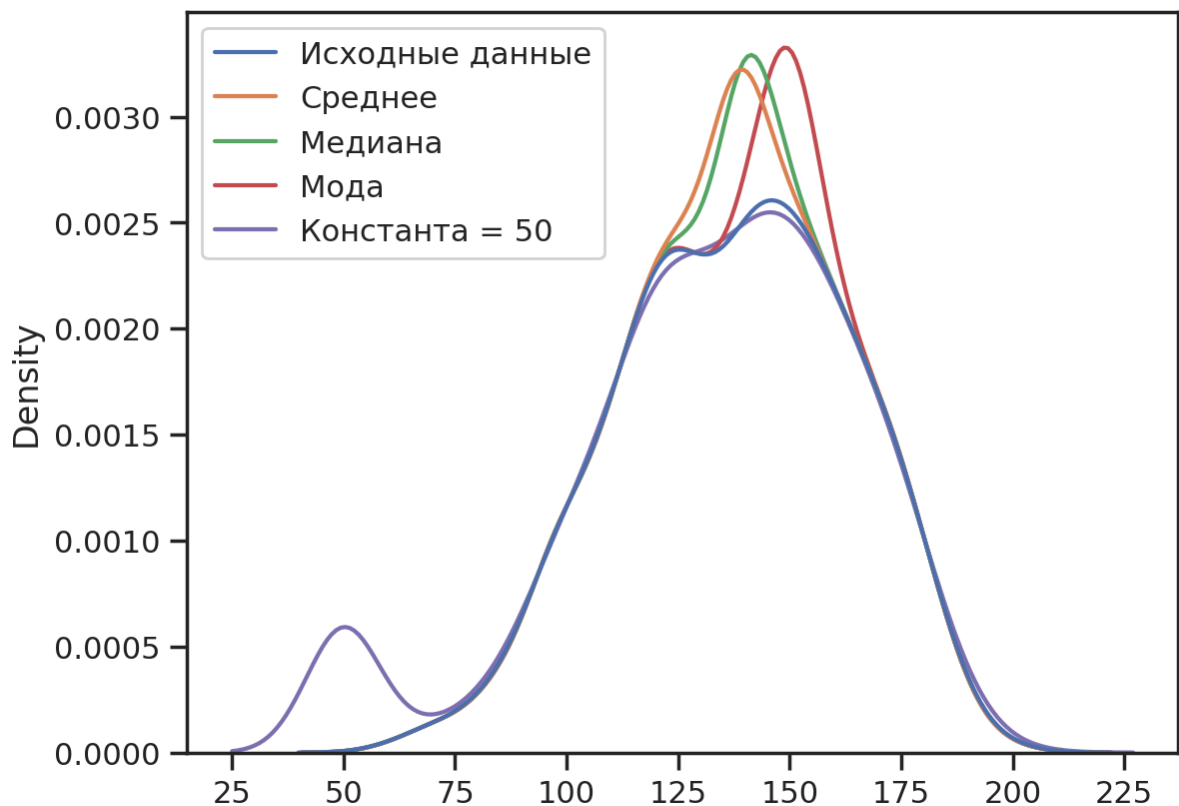
for i in range(len(strategy_params)):
    strategy = strategy_params[i]
    col_name = strategy_params_names[i]
    if (strategy!='constant') or (strategy == 'constant' and const_value!=
        if strategy == 'constant':
            temp_data, _, _ = impute_column(dataset, num_column, strategy,
        else:
            temp_data, _, _ = impute_column(dataset, num_column, strategy)
        new_df[col_name] = temp_data
    sns.kdeplot(data=new_df)

```

```
In [83]: research_impute_numeric_column(data, 'thalch', 300)
```



```
In [84]: research_impute_numeric_column(data, 'thalch', 50)
```



## Заполнение "хвостом распределения"

### Для числовых признаков.

**Допущение:** пропуски распределены НЕ случайным образом. Мы хотим выделить пропущенные значения из остальных значений.

### Преимущества:

- Простота реализации.
- Выделяет пропущенные значения из остальных значений.

### Недостатки:

- Нарушение параметров исходного распределения.
- Поскольку значения на краю распределения фактически являются аномалиями (выбросами), то данный подход может пересекаться с алгоритмами поиска аномалий.

### Как вычислить "хвост распределения"?

Если распределение данных признака  $f$  напоминает нормальное:

$$\text{extreme\_value} = \text{mean}(f) + 3 \cdot \text{std}(f)$$

Для асимметричного распределения:

$IQR = Q3 - Q1$

$IQR$  - \$ interquartile range.

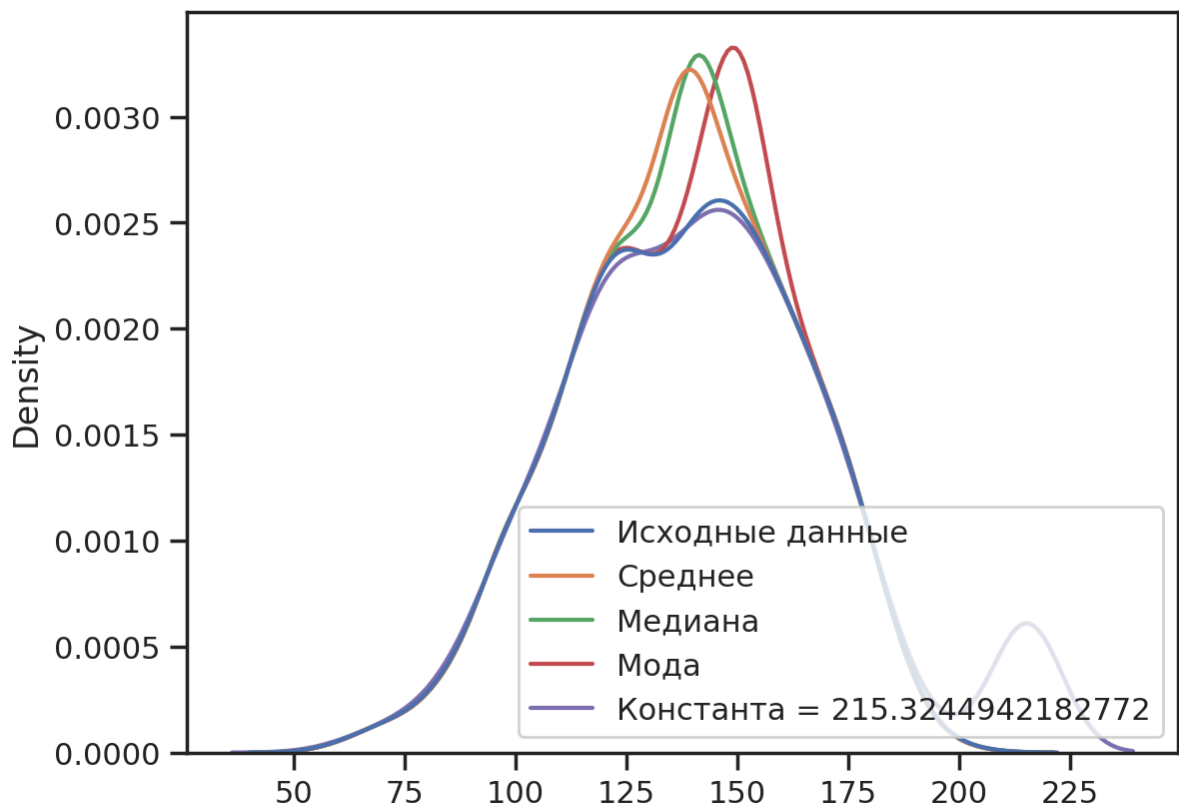
$extreme\_value = Q3 + K \cdot IQR$

Значение  $K$  обычно выбирается равным 1,5. Но для экстремальных выбросов выбирают  $K=3$ .

```
In [85]: # Похоже на нормальное
LotFrontage_ev = data['thlch'].mean() + 3*data['thlch'].std()
LotFrontage_ev
```

```
Out[85]: np.float64(215.3244942182772)
```

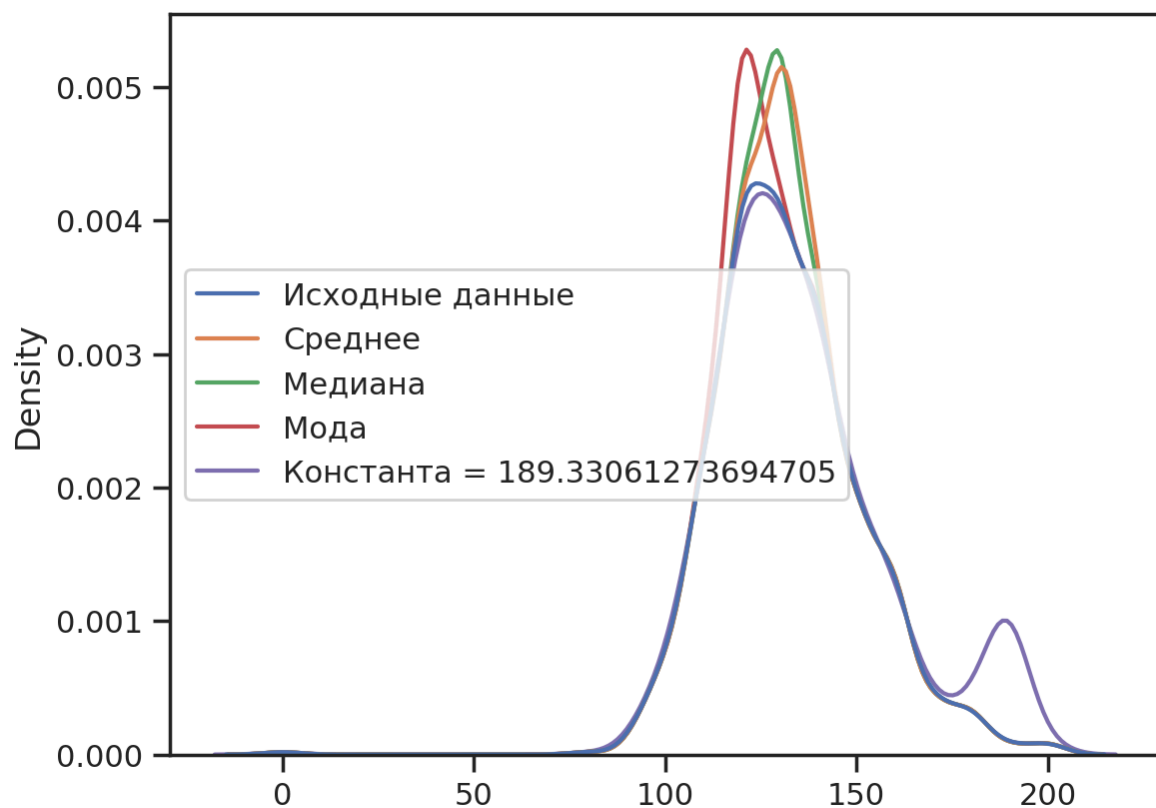
```
In [88]: research_impute_numeric_column(data, 'thlch', LotFrontage_ev)
```



```
In [89]: LotFrontage_ev = data['trestbps'].mean() + 3*data['trestbps'].std()
LotFrontage_ev
```

```
Out[89]: np.float64(189.33061273694705)
```

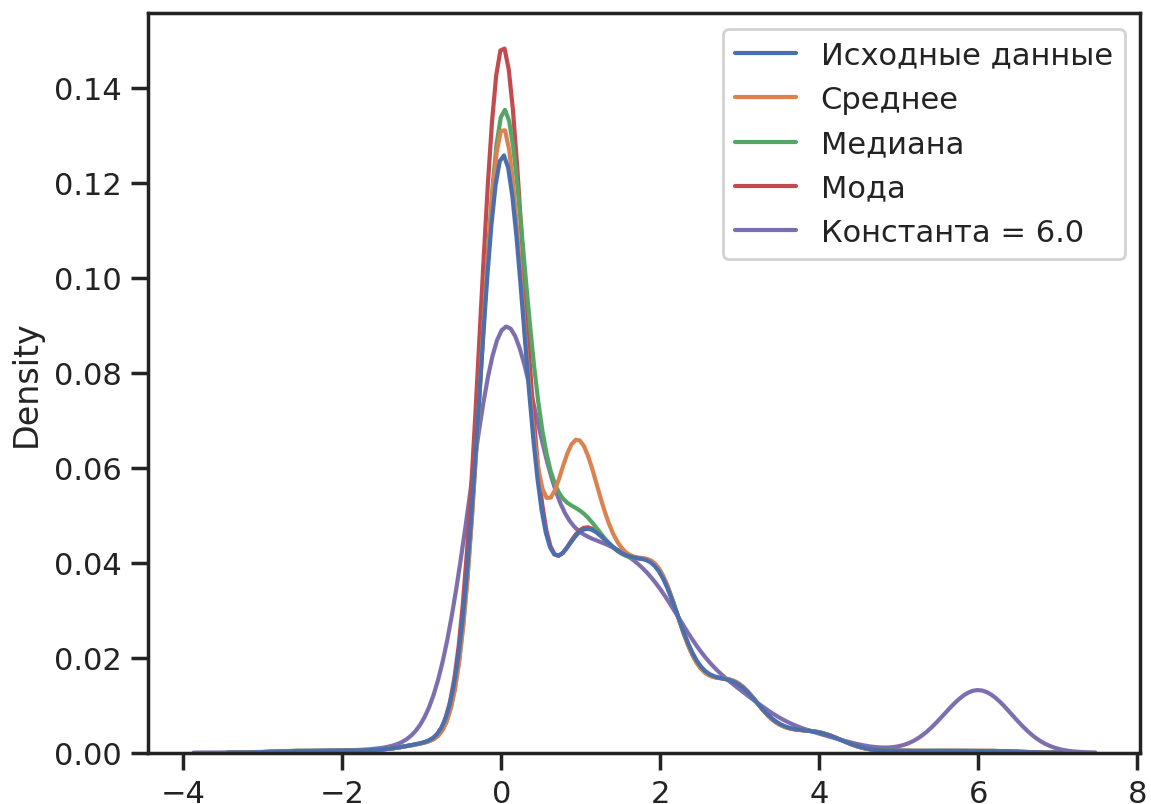
```
In [90]: research_impute_numeric_column(data, 'trestbps', LotFrontage_ev)
```



```
In [91]: # Ассиметричное
IQR = data['oldpeak'].quantile(0.75) - data['oldpeak'].quantile(0.25)
MasVnrArea_ev1 = data['oldpeak'].quantile(0.75) + 3*IQR
print('IQR={}, extreme_value={}'.format(IQR, MasVnrArea_ev1))
```

IQR=1.5, extreme\_value=6.0

```
In [92]: research_impute_numeric_column(data, 'oldpeak', MasVnrArea_ev1)
```



## Обработка пропусков в категориальных данных

Выберем категориальные колонки с пропущенными значениями

Цикл по колонкам датасета

```
In [93]: cat_cols = []
for col in data.columns:
    # Количество пустых значений
    temp_null_count = data[data[col].isnull()].shape[0]
    dt = str(data[col].dtype)
    if temp_null_count > 0 and (dt == 'object') or (dt == 'str'):
        cat_cols.append(col)
        temp_perc = round((temp_null_count / total_count) * 100.0, 2)
        print('Колонка {}'. Тип данных {}. Количество пустых значений {}, {}%.
```

Колонка restecg. Тип данных object. Количество пустых значений 2, 0.22%.  
 Колонка slope. Тип данных object. Количество пустых значений 309, 33.59%.  
 Колонка thal. Тип данных object. Количество пустых значений 486, 52.83%.

```
In [97]: cat_temp_data = data[['slope']]
cat_temp_data.head()
```

Out[97]:

|   | slope       |
|---|-------------|
| 0 | downsloping |
| 1 | flat        |
| 2 | flat        |
| 3 | downsloping |
| 4 | upsloping   |

```
In [98]: cat_temp_data['slope'].unique()
```

Out[98]: array(['downsloping', 'flat', 'upsloping', nan], dtype=object)

```
In [99]: cat_temp_data[cat_temp_data['slope'].isnull()]
```

Out[99]:

|     | slope |
|-----|-------|
| 303 | NaN   |
| 304 | NaN   |
| 305 | NaN   |
| 306 | NaN   |
| 307 | NaN   |
| ... | ...   |
| 915 | NaN   |
| 916 | NaN   |
| 917 | NaN   |
| 918 | NaN   |
| 919 | NaN   |

309 rows × 1 columns

```
In [ ]: # Импутация наиболее частыми значениями
imp2 = sklearn.impute.SimpleImputer(missing_values=np.nan, strategy='most_frequent')
data_imp2 = imp2.fit_transform(cat_temp_data)
data_imp2
```

```
In [101]: # Пустые значения отсутствуют
np.unique(data_imp2)
```

Out[101]: array(['downsloping', 'flat', 'upsloping'], dtype=object)

```
In [ ]: # Импутация константой
```

```
imp3 = sklearn.impute.SimpleImputer(missing_values=np.nan, strategy='constant')
data_imp3 = imp3.fit_transform(cat_temp_data)
data_imp3
```

```
In [103... np.unique(data_imp3)
```

```
Out[103... array(['NA', 'downsloping', 'flat', 'upsloping'], dtype=object)
```

```
In [104... data_imp3[data_imp3=='NA'].size
```

```
Out[104... 309
```

```
In [ ]: data_imp2.tolist()
```

```
In [107... data['slope'] = data_imp2.ravel()
data['slope'].unique()
```

```
Out[107... array(['downsloping', 'flat', 'upsloping'], dtype=object)
```

## Кодирование категориальных признаков

```
In [108... types=data["slope"]
types.value_counts()
```

```
Out[108...
              count
slope
flat          654
upsloping     203
downsloping    63
```

**dtype:** int64

## Кодирование категорий целочисленными значениями - label encoding

```
In [109... le=sklearn.preprocessing.LabelEncoder()
type_le=le.fit_transform(types)
print(np.unique(type_le))
le.inverse_transform(np.unique(type_le))
```

```
[0 1 2]
```

```
Out[109... array(['downsloping', 'flat', 'upsloping'], dtype=object)
```

## Pandas get\_dummies - one-hot быстрый вариант кодирования

```
In [110... type_s=pd.get_dummies(types)
type_s.head(25)
```

```
Out[110... 
```

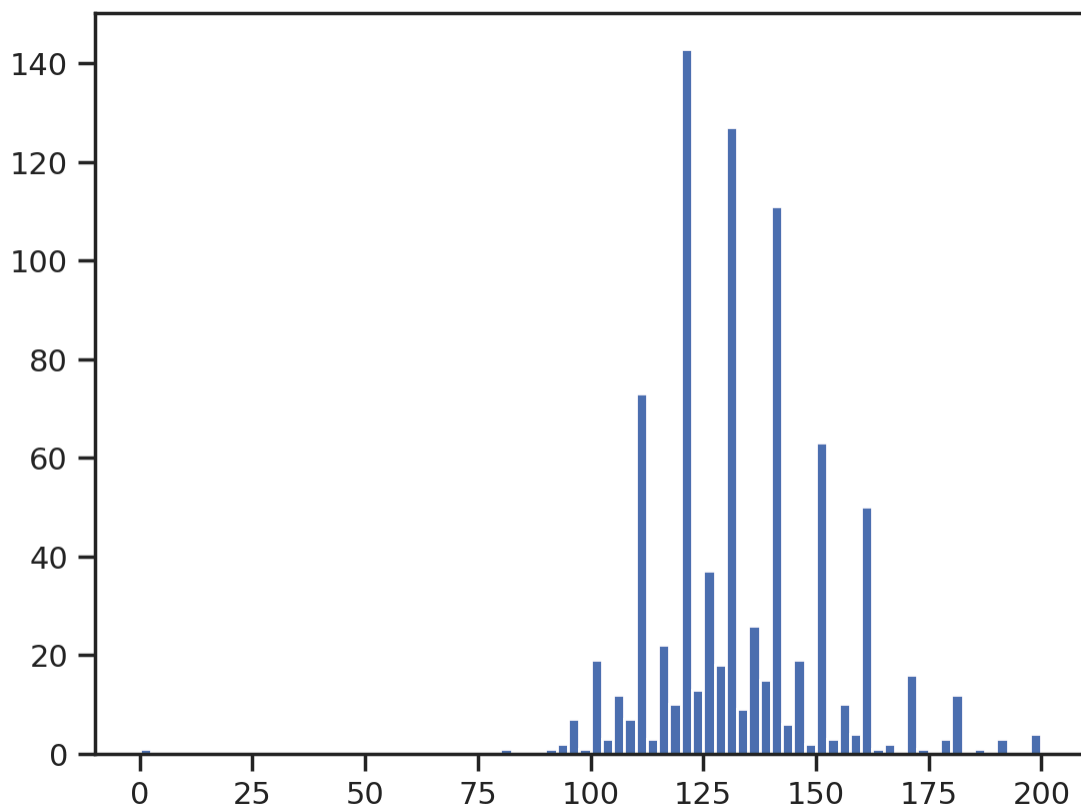
|           | <b>downsloping</b> | <b>flat</b> | <b>upsloping</b> |
|-----------|--------------------|-------------|------------------|
| <b>0</b>  | True               | False       | False            |
| <b>1</b>  | False              | True        | False            |
| <b>2</b>  | False              | True        | False            |
| <b>3</b>  | True               | False       | False            |
| <b>4</b>  | False              | False       | True             |
| <b>5</b>  | False              | False       | True             |
| <b>6</b>  | True               | False       | False            |
| <b>7</b>  | False              | False       | True             |
| <b>8</b>  | False              | True        | False            |
| <b>9</b>  | True               | False       | False            |
| <b>10</b> | False              | True        | False            |
| <b>11</b> | False              | True        | False            |
| <b>12</b> | False              | True        | False            |
| <b>13</b> | False              | False       | True             |
| <b>14</b> | False              | False       | True             |
| <b>15</b> | False              | False       | True             |
| <b>16</b> | True               | False       | False            |
| <b>17</b> | False              | False       | True             |
| <b>18</b> | False              | False       | True             |
| <b>19</b> | False              | False       | True             |
| <b>20</b> | False              | True        | False            |
| <b>21</b> | False              | False       | True             |
| <b>22</b> | False              | True        | False            |
| <b>23</b> | False              | False       | True             |
| <b>24</b> | False              | True        | False            |

# Масштабирование данных

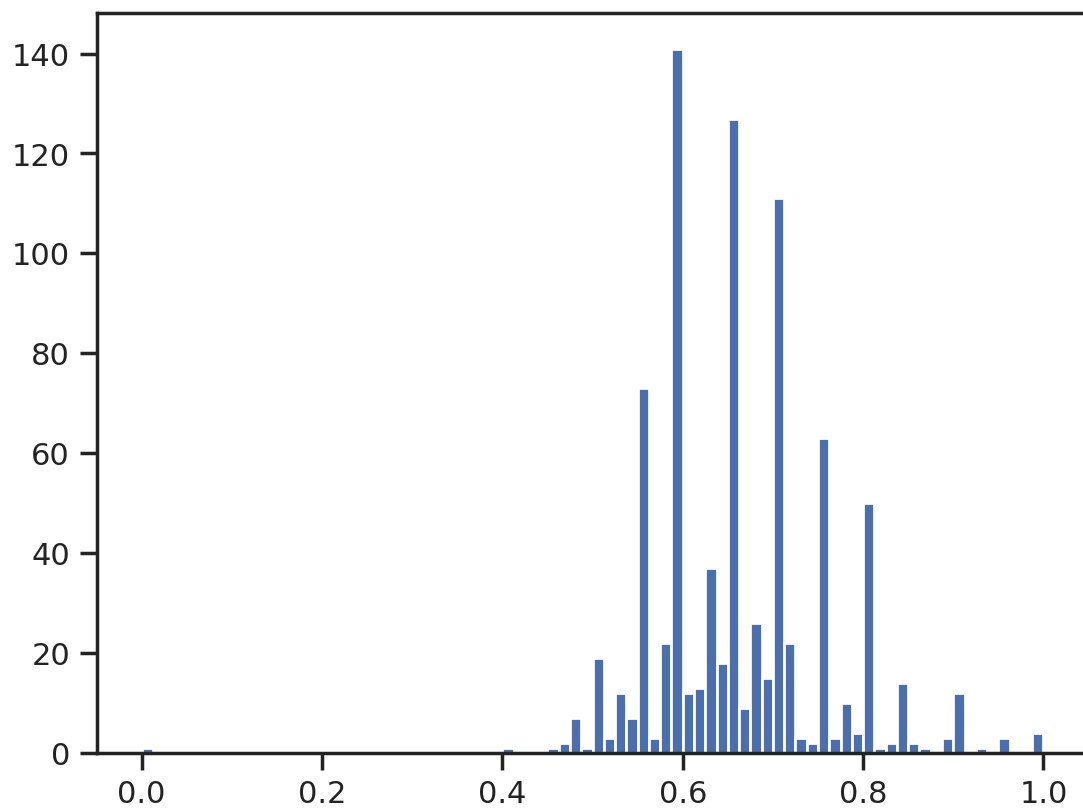
Масштабирование предполагает изменение диапазона измерения величины а нормализация изменение распределения этой величины , - . Если признаки лежат в различных диапазонах то необходимо их , нормализовать Как правило применяют два следующих подхода . , :

## MinMax масштабирование

```
In [111... from sklearn.preprocessing import MinMaxScaler, StandardScaler
sc1 = MinMaxScaler()
sc1_data = sc1.fit_transform(data[['trestbps']])
plt.hist(data['trestbps'], 80)
plt.show()
```



```
In [112... plt.hist(sc1_data, 80)
plt.show()
```



## Масштабирование данных на основе Z-оценки - StandardScaler

```
In [113... sc2 = StandardScaler()  
sc2_data = sc2.fit_transform(data[['trestbps']])  
plt.hist(sc2_data, 80)  
plt.show()
```

